

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN

Análise, deseño e implementación dunha gramola virtual.

Estudante: Carlos Mañoso Sierra
Dirección: Diego Andrade Canosa

A Coruña, August de 2021.

Para todas as persoas que desexan escoitar a súa canción favorita.

Agradecementos

A todos eses que desexaban ver este proxecto rematado, especialmente a Cris e a Sergio.

Resumo

Este proxecto trátase da creación dunha gramola virtual, coma as de sempre pero sen a necesidade de un aparello físico. Con un dispositivo capaz de reproducir música dende Spotify e usuarios con acceso a internet sería suficiente para o seu funcionamento. Trátase dunha aplicación web na que dunha forma moi sinxela se pode crear unha sesión de música onde os usuarios podense unir e engadir as súas cancións favoritas ou votar polas que mais lles gusten.

Abstract

The project it's a virtual jukebox, like the ones at the bars but without the physical machine. With a device capable of playing music over Spotify and users with internet access, it's enough to work. It's a web application where easily you can create a session and users will be able to join then add their favourite songs or vote for the ones that they like.

Palabras chave:

- Gramola
- Música
- Bar
- Cancións
- Spotify
- Web app
- Ionic
- Django

Keywords:

- Jukebox
- Music
- Bar
- Songs
- Spotify
- Web app
- Ionic
- Django

Índice Xeral

1 Introdución **1**

1.1 Motivación . . . 1
1.2 Obxetivo . . . 1
1.3 Estado da arte . . . 1
1.4 Obxetivos . . . 2

2 Tecnoloxías **3**

2.1 Linguaxes . . . 3
2.2 Frameworks e librarías . . . 4
2.3 Ferramentas de desenvolvemento . . . 5
2.4 Sistema de xestión de base de datos . . . 5
2.5 APIs . . . 6

3 Análise **7**

3.1 Metodoloxía . . . 7
3.2 Scrum . . . 7
3.2.1 Roles . . . 7
3.2.2 Reunións . . . 8
3.2.3 Artefactos Scrum . . . 8
3.2.4 Scrum no proxecto . . . 8
3.3 Viabilidade . . . 9
3.3.1 Viabilidade económica . . . 9
3.3.2 Viabilidade técnica . . . 9
3.3.3 Viabilidade do mercado . . . 10
3.4 Análise . . . 10
3.5 Requisitos da aplicación . . . 11
3.5.1 Requisitos funcionais . . . 11
3.5.2 Requisitos non funcionais . . . 11

3.5.3 Actores 12

4 Planificación **13**
4.1 Planificación inicial 13
4.2 Adaptación da planificación a Scrum 13
4.2.1 Primeiro sprint 14
4.2.2 Segundo sprint 14
4.2.3 Terceiro sprint 14
4.2.4 Cuarto sprint 14
4.2.5 Quinto sprint 14
4.3 Custos do proxecto 15

5 Deseño **17**
5.1 Lóxica de funcionamento 17
5.2 Arquitectura 18
5.2.1 MVC: Modelo, vista, controlador 18
5.2.2 MVT: Modelo, vista, plantilla 18
5.2.3 Esquema xeral 19
5.3 Frontend 20
5.3.1 Arquitectura 20
5.3.2 Vista 20
5.3.3 Controller 21
5.4 Backend 24
5.4.1 Modelo 24
5.4.2 Plantilla 25
5.4.3 Vista 25

6 Conclusións e traballo futuro **31**
6.1 Conclusións 31
6.2 Traballo futuro 31

Bibliografía **37**

Índice de Figuras

4.1 Mockups do frontend 15

5.1 Model, Template, View. 19
5.2 Estrutura da base de datos 25

Índice de Táboas

4.1 Tempo empregado no proxecto 15

Capítulo 1

Introdución

NESTE primeiro capítulo da memoria, se explica á motivación, se expoñen os obxectivos e se define á estrutura.

1.1 Motivación

Durante a décadas dos 60 e 70 a industria musical sufriu unha revolución, nunha gran parte dos lugares de restauración comezaron aparecer aparellos que permitían elixir as seguintes cancións no local, as gramolas. Coa dixitalización da música, as gramolas foron desaparecendo e nalgúns casos adaptándose os novos formatos e dixitalizandose. Nos nosos días, coas grandes facilidades que existen e a música na nube é moi complicado atopar un bar no que se permita elixir a seguinte canción, coma antes.

1.2 Obxetivo

O obxetivo principal de esto proxeto é crear unha gramola virtual baseada nun servidor no que tanto o administrador da gramola coma os usuarios dispoñan dunha interface o mais sinxela posible, facendo deste xento mais doada a súa utilización.

1.3 Estado da arte

Existen diferentes tipos de gramolas físicas, similares as antigas pero cun repertorio musical moi amplo e dotadas das maiores novidades. En canto a gramolas sen aparellos físicos controladas polos clientes dende os seus dispositivos, existe un mercado pequeno.

TouchTunes

Expandida por Norte América e parte de Europa é a mais coñecida, dispón dun gran catálogo, sistema de adaptación da música ao local e diferentes ferramentas para o entretemento dos clientes. Ten un sistema de créditos que se poden consumir nos diferentes establecementos nos que está implantada a plataforma. Páxina oficial TouchTunes

AMI

Como directa competidora de TouchTunes, AMI entertainment comezou creando pianos nos que introducindo unha moeda podíase elixir algunha das cancións, posteriormente adicaronse a facer gramolas físicas e nos nosos días seguen coa dixitalización destas. Tamén funciona cun sistema de créditos pra facer soar a música que mais che gusta. Páxina oficial de AMI entertainment

Outloud

A diferenza das anteriores, esta aplicación está mais pensada pra uso particular, pódese descargar en dispositivos Android ou iOS e a través da rede local os usuarios poden xestionar a lista de reprodución. Páxina oficial Outloud

Raveberry

Este proxecto similar a Outloud está pensado pra ser usado dende unha RaspberryPi, o seu punto forte é a integración de multitude de servizos de transmisión de música e permitir os seus usuarios xestionaren a cola de reprodución dun xeito democrácito. Repositorio de Raveberry en Gitub

1.4 Obxetivos

O principal obxetivo do proxecto é crear unha aplicación web para xestionar unha gramola virtual. Preténdese que a interación dos usuarios seña o mais sinxela posible, permita crear unha cola de reproducción e escoitala en calqueira dispositivo con Spotify, para isto desenvolverase un frontend e un backend en Ionic e Django respectivamente.

Capítulo 2

Tecnoloxías

NESTE segundo capítulo explicase as tecnoloxías utilizadas durante o proceso de desenvolvemento.

2.1 Linguaxes

HTML

HyperText Markup Language (HTML) é unha linguaxe de marcado estándar, utilízase na maioría das páxinas webs xa que serve para crear a estrutura destas, en conxunto con JavaScript e Cascading Style Sheets (CSS) permite cambiar a aparencia e a funcionalidade.

JavaScript

JavaScript é unha linguaxe de programación interpretado, permite engadir contido dinámico e interactivo as páxinas web, pode ser inserido en calquera parte do código dunha páxina web.

LaTeX

LaTeX é un sistema de composición de textos de alta calidade. En xeral é usado para crear artigos e documentos técnicos e científicos aínda que pode utilizarse para todo tipo de textos.

Python

Trátase dunha linguaxe de programación interpretado, que fai fincapé na lexibilidade do código. É multiparadigma xa que acepta tanto programación orientada a obxectos, imperativa e funcional.

Sass

Linguaxe de folla de estilos en cascada, é unha linguaxe de script que é traducido a Cascading Style Sheets (CSS) permite xerar de forma automática follas de estilo engadindo características propias de linguaxes de programación coma variables, funcións, etc.

SQL

É unha linguaxe cun dominio específico, é usado para o deseño e mantemento de bases de datos relacionais, particularmente útil usando modelos de datos.

TypeScript

TypeScript é unha liguaxe de programación libre e open source, trátase dun superconxunto de JavaScript, engade a estes tipos estáticos e obxectos baseados en clases, principalmente é usado para o desenvolvemento de aplicacións do lado do cliente.

2.2 Frameworks e librarías

Angular

Angular é un framework para aplicacións web, está desenvolvido en TypeScript, utilízase pra crear aplicacións web dunha soa páxina, co obxectivo do aumento de aplicacións baseadas no navegador con capacidade Modelo Vista Controlador.

Django

Django é outro framework, baseado en Python, facilita a creación de sitios webs complexos, pon énfase na reutilización da conectividade e a extensibilidade dos compoñentes e no desenvolvemento rápido.

Django REST framework

Baseado en Django, permite crear APIs RESTful, o que permite a transferencia de información entre as interfaces e as bases de datos dunha maneira sinxela. Este framework o utilizamos para crear toda a lóxica do backend.

Ionic

Ionic é framework que engloba un conxunto de ferramentas para o desenvolvemento de aplicacións webs progresivas que tamén permite exportar estas para a creación de aplicacións móbiles tanto para iOS coma Android. No proxecto é utilizado na parte do frontend.

Spotipy

Spotipy é unha libraría de Python que permite o acceso completo a API de Spotify, deste xeito é moito mais doado o traballo con ferramentas que utilizan esta plataforma musical.

2.3 Ferramentas de desenvolvemento

NeoVim

NeoVim é un editor de texto na terminal, é un fork de Vim e trata de mellorar este de xeito colaborativo, facendo mais doado o mantemento e maximizando a extensibilidade. Foi utilizado para o desenvolvemento do código e da memoria.

Git

Git é un sistema de control de versións distribuido de código aberto, deseñado para manexar todo tipo de proxectos, caracterízase principalmente pola súa velocidade, rendemento, usabilidade e flexibilidade.

Postman

É un programa que permite o deseño, o simulacro, a depuración e a realización de probas, monitorización e publicacións de APIs dende un único sitio. Neste proxecto usouse para a realización das probas da API do servizo.

Figma

Figma é un editor vectorial baseado na web, serve para facer prototipos e no noso caso para a realización dos mockups.

2.4 Sistema de xestión de base de datos

SQLite

SQLite é un sistema de xestión de base de datos de código libre, é autocontido, non necesita servidor e transaccional. No noso proxecto o usamos para almacenar tanto os usuarios, as sesións e as cancións.

2.5 APIs

Spotify API

Para facilitar o desenvolvemento de aplicacións que se nutran das capacidades de Spotify, existe unha API que permítenos xestionar a maioría as funcionalidades da plataforma como poden ser a procura, a reprodución, a xestión de listas e demais, nos a utilizamos para estas na aplicación.

Capítulo 3

Análise

NESTE terceiro capítulo se fará unha análise da viabilidade do proxecto e, expoñerase a metodoloxía a utilizar.

3.1 Metodoloxía

A forma de traballo consistirá no estudo dos proxectos relacionados, para continuar coa análise, deseño e implementación e probas do proxecto. Utilizarase a metodoloxía Scrum para o desenvolvemento.

3.2 Scrum

Scrum Scrum é un proceso de desenvolvemento áxil. Permite un desenvolvemento rápido e por iteracións, permítenos inspeccionar o software creado e cambiar as preferencias de traballo no lugar dunha planificación e execución completa dende o principio. É un método no que se aplican de maneira regular un conxunto de boas prácticas para traballar de forma colaborativa e obter o mellor resultado posible. Os equipos organizasen entre eles para discutir as funcionalidades e a prioridade destas, deste xeito despois de cada iteración se pode facer unha demostración do produto nese momento.

3.2.1 Roles

Os roles da metodoloxía son os seguintes:

- Product Owner: é o encargado de fixar as metas do sprint e a priorización do Product Backlog. O seu obxectivo é que o produto maximice o seu valor.
- Scrum Master: a súa responsabilidade é promover as prácticas Scrum, media entre o equipo de desenvolvemento e o Product Owner. Dirixe as reunións diarias e as revisións

dos Sprints.

- Equipo de desenvolvemento: organizanse para desenvolver o proxecto seguindo o Sprint Backlog.

3.2.2 Reunións

- Daily Scrum: tratase dunha reunión diaria duns 15 minutos de duración que fai o equipo de desenvolvemento, é dirixida polo Scrum Master e nela cada participante responde a 3 sinxelas preguntas. 'Qué fixeches onte?´, 'Qué vas facer hoxe?´, 'Qué problemas atopaches?´, deste xeito pódese ver o estado actual do proxecto.

- Sprint Planning: nesta reunión se seleccionan as tarefas a realizar durante o Sprint, se fai unha estimación da súa duración e se obtén o Sprint Backlog, participan tanto o equipo de desenvolvemento coma o Scrum Master.

- Sprint Review: ao final de cada Sprint realizase esta reunión que ten como obxectivo presentar os traballos realizados neste, poden realizarse adaptacións do Product Backlog.

- Sprint Retrospective: tamén se realiza ao final de cada sprint e ten como finalidade analizar o traballo realizado, as funcionalidades e as melloras.

Unha vez rematadas tanto o Sprint Review e o Sprint Retrospective iniciase a planificación do seguinte Sprint Planning de forma cíclica.

3.2.3 Artefactos Scrum

- Product Backlog: é a lista que recolle todos os requisitos do produto, ordenado de maior a menor prioridade. Trátase dunha lista dinámica xa que vai cambiando cos diferentes Sprints.

- Sprint Backlog: agrupa en tarefas os elementos do Product Backlog seleccionados para o Sprint actual.

3.2.4 Scrum no proxecto

Neste proxecto, debido as diferencias coa realidade se fixeron modificacións da metodoloxía. Coma non hai un equipo de desenvolvemento, os roles de Scrum Master e o equipo de desenvolvemento foron manexados polo alumno e o rol de Product Owner polo director.
Cada Sprint Planning tivo unha duración de aproximadamente dúas semanas, o Daily Scrum foi substituído por un repaso das tarefas realizadas e un diario de traballo e o Sprint Planning

realizado de forma individual.
Para a creación do Product Backlog se realizaron reunións ao principio do desenvolvemento para a concreción dos aspectos fundamentais da aplicación, foi cambiando co paso do tempo segundo as versións funcionais do produto.
Os Sprint Backlog foron seleccionados polo alumno segundo as prioridades establecidas inicialmente, ademais de seguir unha orde lóxica de desenvolvemento de funcionalidades, dende as mais básicas ata as específicas.
Tanto o Sprint Review coma o Sprint Rertospective reducidas debido a complexidade.

3.3 Viabilidade

Nesta parte pasaremos a expoñer a análise de viabilidade do proxecto, algo esencial antes de comezar calquera tipo de proxecto xa que determinaremos se poderemos acadar a súa finalización satisfactoriamente.

3.3.1 Viabilidade económica

No desenvolvemento da maioría dos proxectos software existen tres custos principais, os custos en recursos humanos, en software e en hardware.
Os recursos humanos neste caso ao ser unha soa persoa a encargada de facer o deseño, implementación e probas o custo é baixo, noutros proxectos onde hai mais persoas implicadas habería que ter en conta o custo por hora de cada unha das persoas involucradas.
Con respecto aos recursos software neste proxecto utilizase software libre na maior medida posible, partindo do sistema operativo ata os frameworks e librarías, pasando polas ferramentas de desenvolvemento coma o editor de texto ou o navegador, todo isto fai que os custos software neste proxecto sexan nulos.
En canto aos recursos hardware, para este proxecto non foi necesario adquirir nada, xa que o único necesario é un ordenador con conexión a Internet, podería ser necesario un servidor na nube para despregar a aplicación o completo, pero debido ao pequeno tamaño da aplicación poderíase recurrir a un servizo gratuíto se fose necesario.
Polo tanto respecto a viabilidade económica podemos dicir que este proxecto é completamente factible xa que os custos vense moi reducidos polas características deste.

3.3.2 Viabilidade técnica

Como xa vimos, a maioría de software é altamente coñecido no mundo do desenvolvemento software e dispón de moita documentación, polo tanto a realización do proxecto non presenta unha dificultade técnica alta. Podemos afirmar entón que o proxecto tamén é viable tecnicamente.

3.3.3 Viabilidade do mercado

Hoxe en día podemos ver diferentes tipos de software comercial que reproduce gramolas virtuais para establecementos, estes atópanse algo implantados en Norte América e comezan a despregarse algo en Europa, en canto a software libre non hai opcións comerciais, tan só algúns proxectos para implementar de forma local, incluso Spotify ten unha función de crear unha sesión, pero que non é tan doado de utilizar. Non foron atopadas aplicacións similares de código aberto. Este software pode atoparse no apartado estado da arte 1.3. Por este motivo e os anteriores comentados podemos afirmar que este proxecto ten cabida no mercado, entón podemos consideralo coma viable.

3.4 Análise

Para a creación deste proxecto se van desenvolver un frontend baseado en Ionic no que se amosarán todos os datos relacionados coas cancións que están na cola, o número de votos e a súa orde de reprodución. Tamén se vai realizar un backend en Django Rest Framework que nos permitirá a conexión do frontend coa API de Spotify e así levar a lóxica do proxecto a cabo. As funcións destacadas son as seguintes:

- Creación dunha sesión.
  Poderase crear unha sesión onde se vai reproducir a música, será necesario facer a autenticación contra a API de Spotify cunha conta premium. Esta sesión pode interpretarse como sería o bar ou lugar de emprazamento dunha gramola física, pero por suposto que pode ser usado en oficinas, reunións particulares e calquera sitio onde a música teña cabida.

- Creación de usuarios.
  Os usuarios poderán rexistrarse nas sesións, o primeiro en facelo terá o rol de administrador, deste xeito terán a posibilidade de engadir cancións a cola de reprodución e votar a orde destas.

- Creación dunha cola de reprodución.
  Os usuarios poderán engadir cancións a unha cola de reprodución xenérica para a sesión, a aplicación encargarase de que cando remate unha canción comece a reprodución da seguinte.

- Posibilidade de facer votacións.
  Na cola de reprodución aparecen o número de votos de cada canción e botóns que sirven para engadir ou substraer votos, a lista de reprodución estará ordenada por este número de votos e posteriormente por orden de chegada, coma sería nunha gramola habitual.

- Poboar a cola de reprodución cunha lista xa creada.
  Aproveitando o potencial das listas de reprodución que ten Spotify, a cola de reprodución poderá ser enchida cunha playlist do usuario, o obxectivo de esta funcionalidade é que os usuarios poidan votar a orde de reprodución das cancións da lista.

3.5 Requisitos da aplicación

Os requisitos son as necesidades funcionais que ten un produto para satisfacer o seu obxectivo. Os requisitos poden ser funcionais ou non funcionais, os primeiros son os que describen o comportamento de funcións particulares no software, os segundos non están relacionados especificamente coa funcionalidade do sistema.

3.5.1 Requisitos funcionais

- Amosar información da cola de reprodución.
- Amosar os resultados de procura.
- Engadir cancións a cola.
- Permitir a creación de sesións.
- Permitir as votacións aos usuarios.
- Permitir facer unha procura de cancións.
- Permitir o rexistro de usuarios.
- Facer votacións positivas ou negativas.

3.5.2 Requisitos non funcionais

- **Deseño:** minimalista fácil de comprender e sinxela de usar.
- **Consistencia:** O modelo da aplicación mantense actualizado sen perder datos.
- **Seguridade:** Tan só os usuarios rexistrados poden facer uso da aplicación.
- **Fiabilidade:** Os erros en produción están controlados para poder dar unha resposta ó usuario en caso de que se dean.

3.5.3 Actores

No proxecto existen tres tipos de usuarios, usuarios sen rexistrar, usuarios rexistrados e administrador.

- **Usuario sen rexistrar:** O usuario sen rexistrar ten acceso a páxina de inicio dende onde se pode crear unha nova sesión ou facer login.
- **Usuario rexistrado:** Os usuarios rexistrados poden engadir ver a cola de reprodución, buscar e engadir cancións e votar positiva ou negativamente as cancións.
- **Usuario administrador:** O usuario administrador pode facer todo o que os usuarios rexistrados e ademais pode poboar a cola de reprodución con listas de reprodución de Spotify, tamén é o usuario que engade o token a sesión.

Capítulo 4

Planificación

Para a planificación dos proxectos se deben ter en conta tanto os custos como o tempo, neste capítulo explicase as etapas da desta durante o proceso de desenvolvemento, a planificación é acorde a metodoloxía Scrum 3.2.

4.1 Planificación inicial

Unha vez realizado a análise (capítulo 3) se decidiu dividir o traballo en 5 grupos:

- **Configuración inicial do proxecto:** Recolección de información sobre as ferramentas a utilizar para a posta en marcha do proxecto, configuración do entorno de desenvolvemento.
- **Implementación da persistencia:** Deseño da base de datos, modificación do modelo de usuarios de Django.
- **Conexión coa API de Spotify:** Creación da lóxica no backend, peticións necesarias da aplicación o API de Spotify. Configuración da xestión de usuarios.
- **Conexión frontend backend:** Planificación e implementación do frontend, conexión con JSON web Token (JWT) aos usuarios do backend.
- **Xeración do espazo web:** Peticións ao backend e xestión de permisos.

4.2 Adaptación da planificación a Scrum

O desenvolvemento do proxecto se dividiu en 4 fases, as cales tiveron unha duración de entre 3 e 5 semanas dependendo da complexidade, a metodoloxía foi adaptada a natureza do proxecto. Se realizou un Sprint Planning 3.2.2 ao principio de cada fase onde se obtivo o

Sprint Backlog 3.2.3 e ao finalizar cada unha se fixo o Sprint Review 3.2.2 e o Sprint Retrospective 3.2.2. Durante a realización do proxecto, tamén se levou un diario de traballo que facilitou a organización.

4.2.1 Primeiro sprint

No primeiro sprint se levou a cabo a implementación a primeira fase, se creou o Product Backlog 3.2.3 baseado nos requisitos funcionais. Posteriormente se estableceu o espazo de traballo, se configurou o editor NeoVim cunha selección de plugins adaptados ao proxecto, tamén se configurou o repositorio de GitLab. Neste sprint tamén incluímos as probas iniciais tanto coa API de Spotify coma con Ionic xa que se trataba dun framework novo para o desenvolvedor, por último tamén se realizaron os mockups do frontend 4.1.

4.2.2 Segundo sprint

Neste sprint se deseñou a base de datos, se crearon os modelos de datos en Django e se modificou o modelo xa preestablecido de usuarios en Django para engadir mais datos que este non contiña, na figura 5.2 podemos ver a súa estrutura.

4.2.3 Terceiro sprint

No terceiro sprint fíxose a conexión do backend coa API de Spotify, crearonse as peticións necesarias coa axuda da libraría de Python Spotipy, esta proporciona todas as funcionalidades necesarias, dende a obtención do token de autenticación contra a plataforma de música ata a reprodución, pasando pola xestión de playlists e a cola de reprodución.

4.2.4 Cuarto sprint

Para o cuarto sprint se planificou o deseño do frontend baseado nos mockups realizados. Tamén se fixo a conexión mediante JWT para a xestión de usuarios. Estudáronse outros métodos para facer dita conexión, pero se concluíu que a autenticación con JWT era a mais segura e útil no noso caso. Este sprint foi mais longo do esperado debido a novidade no uso deste tipo de autenticación, poñendo fincapé na especial dificultade de renovar os tokens de autenticación co token de refresco.

4.2.5 Quinto sprint

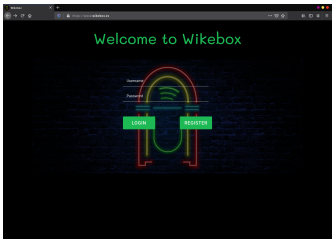
No quinto e último sprint se crearon as páxinas do frontend, se tivo en conta os permisos de acceso tanto na parte do frontend coma do backend permitindo tan só as accións necesarias.

4.3 Custos do proxecto

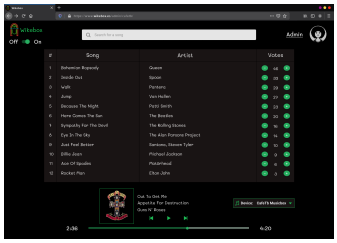
O software utilizado durante a creación deste proxecto foi sempre con licencias libres, en canto a recursos hardware non foron necesarios debido a que tan só foi necesario un ordenador do que xa se dispoñía. Polo tanto os recursos utilizados restrínxese tan só os recursos humanos.

Na táboa 4.1 mostrase os recursos humanos utilizados medidos en horas de traballo.

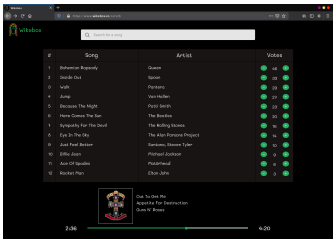
Figura 4.1: Mockups do frontend



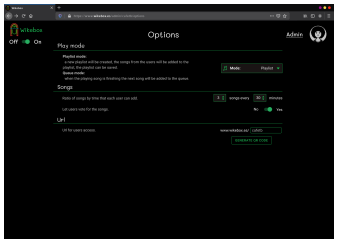
(a) Mockup vista de login



(b) Mockup vista do admin



(c) Mockup vista de usuario



(d) Mockup opcións do admin

Táboa 4.1: Tempo empregado no proxecto

Tarefa	Tempo
Documentación	80h
Desenvolvemento do backend	100h
Desenvolvemento do frontend	200h
Probas	30h
Total	410h

Capítulo 5

Deseño

No seguinte capítulo explicáranse os aspectos relevantes do deseño da nosa aplicación, coma a arquitectura adoptada e os servizos utilizados.

5.1 Lóxica de funcionamento

Esta aplicación está deseñada coa idea de reproducir unha gramola das que existían habitualmente nos negocios de restauración pero de forma virtual, engadindo ademais algunhas características que creemos que poden ser útiles e facer o funcionamento desta máis atractivo para os usuarios finais. Coma é razoable non nos gustaría que a aplicación estivese deseñada nun nicho tan concreto coma o da restauración, polo tanto que sexa útil noutros ámbitos nos que se desexe crear unha lista de reprodución compartida sen complicación coma pode ser un ambiente de traballo cun fío musical, unha festa ou similares.
Ademais tamén sería desexable que a aplicación resultase sinxela para o usuario final, polo tanto para un novo usuario os pasos normais a seguir serían:

1. Creación dunha nova instancia ou sesión (poderíamos usar o símil dun bar).
2. Creación dun usuario nesta sesión, este ao ser o primeiro será o usuario administrador.
3. Conexión da sesión co usuario de Spotify.
4. Reproducir música en calquera dos clientes de Spotify, xa pode ser a web, os clientes de escritorio ou móbil.
5. Compartir o enlace da sesión para o resto dos usuarios.

Para o caso de novos usuarios nunha sesión xa rexistrada, poderíamos usar o símil de os clientes nun bar os pasos serían os seguintes:

1. Abrir o enlace recibido, que te achega a páxina de novo usuario dentro da sesión.

2. Rexistrar o novo usuario.

3. Engadir cancións e votar polas favoritas para que suban na cola de prioridade.

5.2 Arquitectura

No caso deste proxecto utilízanse dous frameworks diferentes no que cada seu teñen unha arquitectura diferente, Angular (o framework que usamos con Ionic) utiliza a típica modelo, vista, controlador e Django cambia esta derradeira por plantilla[1]. Ámbolos patróns de arquitecturas se basean nas ideas de reutilización do código e separación de conceptos o que separa as representacións internas da información co xeito que se presenta os usuarios finais da aplicación, estas características son básicas para facilitar a tarefa de desenvolver e facer mais doado o mantemento, pero o seu funcionamento non e de todo igual.

5.2.1 MVC: Modelo, vista, controlador

Trátase do patrón de deseño software máis coñecido, é moi utilizado no desenvolvemento web.

- **Model:** O **modelo** é a representación da información coa que o sistema opera, polo tanto xestiona todos os accesos a dita información, tanto consultas como actualizacións. Envía a vista a información que esta lle vai solicitando para ser mostrada na interface do usuario, as peticións de acceso ou manipulación chegan ao modelo a través do controlador.

- **View:** A **vista** encargase da representación de cara ao usuario de toda a información da aplicación, realiza as peticións ao modelo, necesarias para a recuperación de dita información. Proporciona unha experiencia interactiva do lado do cliente para o usuario, o que lle permite interactuar coa aplicación.

- **Controller:** O **controlador** é o encargado de interpretar as instrucións e accións que realizan os usuarios finais da aplicación, facendo uso das funcionalidades implementadas no modelo, validando os datos e os parámetros das solicitudes recibidas. Pódese dicir que o controlador fai de intermediario entre a vista e o modelo.

5.2.2 MVT: Modelo, vista, plantilla

A diferencia da arquitectura **MVC,** neste caso a parte do controlador é manexada directamente polo framework, é un patrón de deseño coñecido por Django.

[1] Template en inglés.

- **Model:** O **modelo** é similar ao caso de MVC, actúa como interface para os datos, é basicamente a estrutura lóxica de almacenamento de datos detrás da aplicación web ou a API no noso caso.
- **View** A **vista** executa a lóxica de negocio, fai iteración co modelo e renderiza a plantilla. Acepta peticións HTTP e responde tamén peticións HTTP. É similar ao controller explicada na arquitectura MVC.
- **Template:** A **plantilla** é o compoñente que marca a diferencia coa arquitectura MVC. A plantilla actúa como a capa de presentación, basicamente é o código HTML que renderiza a información. O contido das plantillas pode ser tanto estático coma dinámico, poderíase comparar a vista na arquitectura MVC.

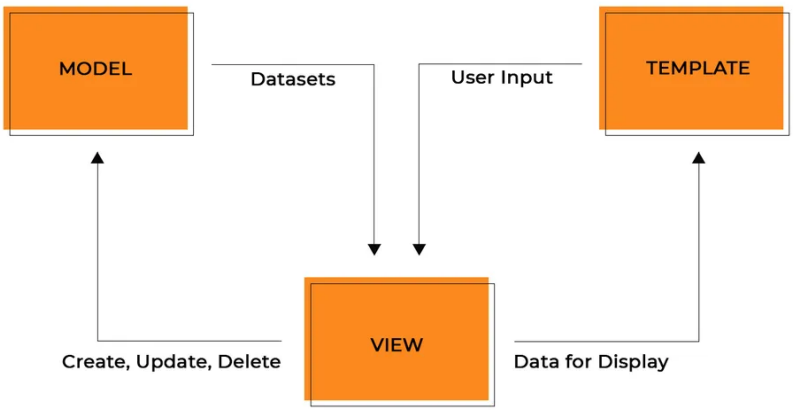


Figura 5.1: Model, Template, View.

Como vemos a gran vantaxe das arquitecturas é a separación das responsabilidades dentro da aplicación, deste xeito se existen cambios nos datos (Modelo) non afecta a visualización xa que esta se actualizará co contido novo, ademais de que o usuario non interactúa en ningún momento directamente cos datos.

5.2.3 Esquema xeral

O esquema xeral para o proxecto é o uso de Ionic para o Frontend e Django para o Backend, a elección destes Frameworks é: Ionic pola súa capacidade de desenvolver con tan só un código tanto unha aplicación web coma aplicacións específicas para os sistemas operativos mais usados nos móbiles hoxe en día, ademais do seu cliente que facilita a xestión de novos recursos ou o manexo destes. No caso de Django pola súa simpleza ao usar Python coma linguaxe de programación, o que simplifica o uso de librerías para a conexión con Spotify por exemplo, ademais de todas as características deste que facilitan o desenvolvemento do código

coma poden ser: o seu propio servidor para manexo do backend, que fai mais doadas as probas nos primeiros estadios da aplicación, ou o framework para as unit test de Python.

5.3 Frontend

O frontend en Ionic permite o uso de diferentes frameworks para o desenvolvemento, no noso caso utilizamos Angular xa que é ideal para a creación de Progressive web applications (PWA) que é a principal idea no noso proxecto. Ademáis de que permítenos utilizar a programación reactiva, que a súa vez está enfocado no patrón de deseño observer que se basea na notificación duns obxectos a outros cando se producen cambios.

- **Programación reactiva:** trátase dun paradigma enfocado ao traballo con fluxos de datos, finitos ou infinitos de forma asíncrona. Ao ser asíncrona a recepción dos datos non fai bloqueos o que facilita a as chamadas a recursos lentos coma poden ser os que están en rede, chamadas a bases de datos, xestión de peticións e en xeral todo o fluxo de chamadas.

- **Observables:** é un modo de implementación da programación reactiva, reacciona ante o fluxo de distintos eventos. Consta de tres compoñentes principais: **Observable**, o que queremos observar, implementado mediante unha colección de eventos ou valores. **Observer**, é o actor que se dedica a observar, permite escoitar os eventos emitidos polos observables. **Subject**, o emisor de eventos, crea un fluxo cando un observable sufre cambios, estes eventos son consumidos polos observables.

5.3.1 Arquitectura

Coma indicamos anteriormente, Ionic segue o a arquitectura de MVC no noso caso tan só utilizamos o modelo para a comprobación dos JWT e para a transferencia de cancións dende o backend e recibir estas de forma correcta mediante un observable, para amosalo na vista. Na sección 5.4.1 explícase o modelo do backend que é o realmente utilizado na aplicación.

5.3.2 Vista

A vista é a última capa da arquitectura do sistema é a capa que interactúa co usuario mostrándolle a información e enviando as peticións do usuario ao controlador. Ao tratarse dunha aplicación web, a vista execútase nun navegador, ou nun dispositivo móbil coma PWA, polo tanto son páxinas HTML modificadas por JavaScript, codificado en TypeScript no noso caso grazas a Angular.

A continuación vaise expoñer brevemente a estrutura das vistas do frontend que se compón das páxinas en HTML:

- **Home:** trátase da páxina de destino para usuarios non conectados, nela explícase brevemente o funcionamento da aplicación e facilita o acceso as páxinas útiles para os usuarios. No caso dos usuarios xa conectados existe unha redirección a páxina de *Music* xa que se supón que os usuarios xa visitaron a páxina *Home*.
- **Instance:** nesta páxina os usuarios poden crear novas sesións para crear listas de reprodución conxuntas.
- **Login:** permite aos usuarios conectarse a súa sesión, existen páxinas de login para cada sesión, pero tamén é posible o acceso dende a páxina de login xenérica.
- **Music:** a páxina de music é a páxina principal da aplicación, nela os usuarios poden buscar cancións, engadilas a cola, visualizar a canción que está a soar no momento e votar as cancións para cambiar a súa orden. E no caso dos usuarios administradores, tamén poden seleccionar o dispositivo no que se reproduce a música e escoller unha lista de reprodución propia do seu Spotify para encher a aplicación e que os usuarios poidan votar a orde.
- **Register:** permite aos usuarios rexistrarse nunha sesión, cada sesión ten a súa páxina de rexistro, polo tanto os usuarios se rexistran directamente en dita sesión. Esta é a páxina que os usuarios administradores compartirán cos usuarios para permitirlles acceder a súa sesión.
- **Callback:** trátase da páxina a que redirixe Spotify despois de autenticarse na súa plataforma, esta captura o token do usuario.

5.3.3 Controller

O controller é a capa intermedia da arquitectura, actúa coma comunicación entre a vista e o modelo, no noso caso a maioría desta comunicación realizase no modelo, polo tanto moitas das instrucións na parte do controlador do frontend son peticións ao backend. Esta capa dividiuse en tres servizos que se describen a continuación:

- **Api:** este servizo componse de chamadas a API creada no backend, destaca a utilización de observables para a recepción dos datos poñemos de exemplo a función *queueSongs* 5.3.3 que recibe no observable a lista de cancións que se atopan nese momento na cola para ser votadas. Tamén cabe destacar a función *spotifyLoginUrl* 5.3.3 que abre unha nova pestana no navegador para que o usuario poida abrir a súa sesión en Spotify para

posteriormente capturar en *spotifyCallback* 5.3.3 a url de resposta e gardar o token. En tódalas funcións envíase a request xa que permite saber cal é o usuario que está facer a petición, este é o motivo de que as funcións sexan post e non haxa ningún get.

```
queueSongs(): Observable<IEsongs[]>{
  return this.httpClient.post<IEsongs[]>(this.SERVER_ADDRESS +
    'api/queuesongs/', {request: Request});
}
```

```
spotifyLoginUrl() {
  this.httpClient.post(this.SERVER_ADDRESS + 'api/token/',
    {'request': Request}).subscribe((response) => {
    window.open(response[0].url, '_self');
  })
}
```

```
  spotifyCallback() {
    this.httpClient.post(this.SERVER_ADDRESS + 'api/callback/',
    {'url': window.location.href })
  }
```

- **Auth:** o servizo de auth manexa a autenticación dos usuarios, usa JSON web Token para o control de acceso xa que permite a propagación de identidades e privilexios entre o provedor de información e o solicitante. Para o almacenamento dos JWT utilizase Ionic Storage xa usa o mellor método para o gardado de datos en cada plataforma utilizada. A función de *login* 5.3.3 é unha mostra do uso do almacenamento do token usando o storage.

```
login(username: String, password: String) {
  this.user = username
  return this.httpClient.post(this.SERVER_ADDRESS + 'users/token/',
    {
    "username": username,
    "password": password
  })
  .pipe(
    tap(token => {
      this.getInstance()
      this.token = token;
      this.storeJwtToken(this.token.access)
      this.storeRefreshToken(this.token.refresh)
      this.storeUser()
      this.storeExpiration()
      this.storage.set("token", this.token)
      .then(
```

```
17          () => {
18            console.log('Token Stored');
19          },
20          error => console.error('Error storing item', error)
21        );
22        return token;
23      }),
24    );
25  }
```

- **Interceptor:** para o uso de autenticación con JWT é necesario engadir o token en cada unha das peticións ao backend *intercept* 5.3.3 é a función que se encarga de interceptar a petición, a función *addToken* 5.3.3 é a encargada de engadir o token a cabeceira. Para garantir a seguridade os tokens de acceso caducan cada 5 minutos polo tanto necesitamos renovalos co token de refresco como podemos observar a función *handle401Error* 5.3.3 renova o token cando se produce un erro de autenticación.

```
1  intercept(request: HttpRequest<any>, next: HttpHandler):
       Observable<HttpEvent<any>> {
2
3    if (this.authService.getJwtToken()) {
4      request = this.addToken(request,
       this.authService.getJwtToken());
5      console.log(request)
6    }
7
8    return next.handle(request).pipe(catchError(error => {
9      if (error instanceof HttpErrorResponse && error.status === 401)
       {
10       return this.handle401Error(request, next);
11     } else {
12       return throwError(error);
13     }
14   }));
15 }
```

```
1  private addToken(request: HttpRequest<any>, token: string) {
2    return request.clone({
3      setHeaders: {
4        'Authorization': `Bearer \${token}`
5      }
6    });
7  }
```

```
private handle401Error(request: HttpRequest<any>, next:
    HttpHandler) {
  if (!this.isRefreshing) {
    this.isRefreshing = true;
    this.refreshTokenSubject.next(null);

    return this.authService.refreshToken().pipe(
      switchMap((token: any) => {
        this.isRefreshing = false;
        this.refreshTokenSubject.next(token.access);
        return next.handle(this.addToken(request, token.access));
      }));

  } else {
    return this.refreshTokenSubject.pipe(
      filter(token => token != null),
        take(1),
      switchMap(access => {
        return next.handle(this.addToken(request, access));
      }));
  }
}
```

5.4 Backend

Aínda que foi deseñado coa idea de publicar noticias dun xornal, Django é un dos frameworks mais populares hoxe en día, segue o principio (DRY, Don't Repeat Yourself) [2] polo que evita a repetición de código innecesariamente. Está escrito en Python, que axuda a súa popularidade, ademais como xa explicamos utiliza o a arquitectura **MVT** que destaca pola simpleza a hora de cambiar a forma de visualizar o contido.

5.4.1 Modelo

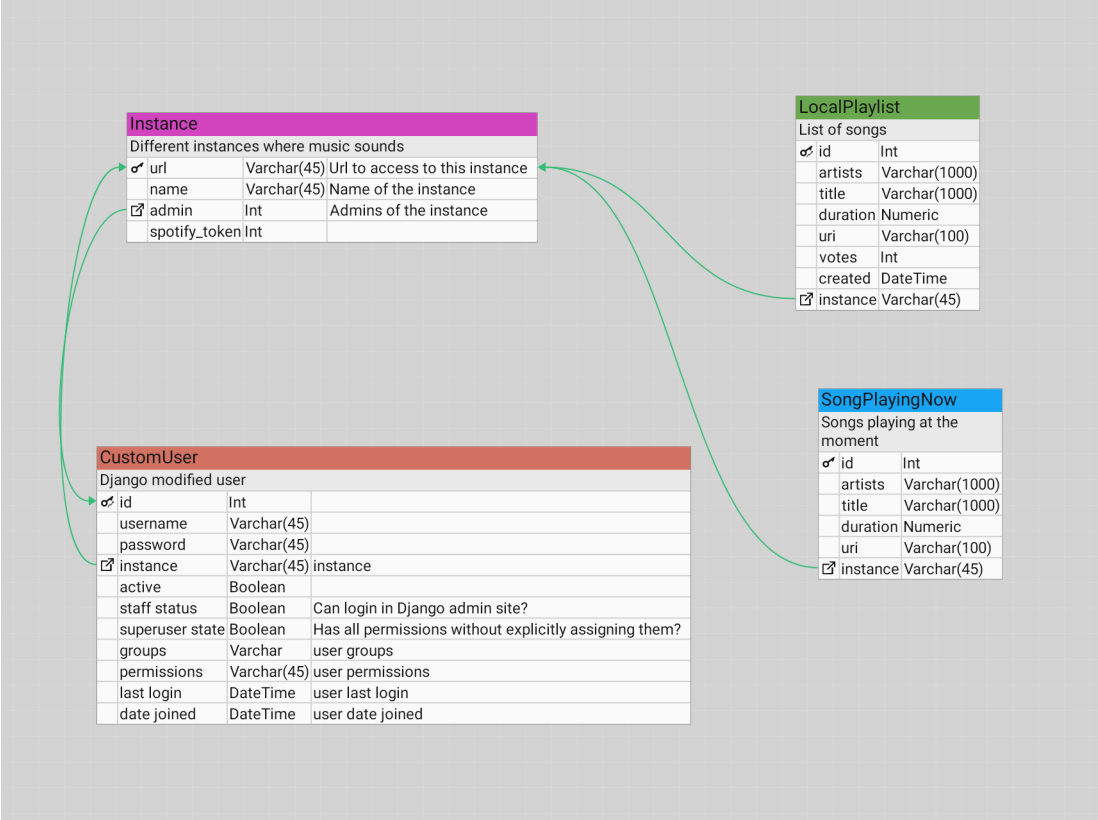
A capa modelo é a parte da aplicación que encapsula a lóxica de negocio cos accesos a base de datos. Nesta capa se almacea, modifica e eliminan os datos que manexa a aplicación. Na figura 5.2 pódense ver as entidades do modelo. A continuación detallase brevemente cada unha das entidades.

- **Instance:** representa os datos das sesións, almacenase a lista de sesións que estean rexistradas na aplicación.

[2] Non te repitas

- **LocalPlaylist:** representa a lista de cancións na cola das sesións.
- **SongPlayingNow:** representa a canción que se está a reproducir nas sesións.
- **CustomUser:** representa os usuarios da aplicación, é unha modificación do modelo User de Django adaptado para o proxecto

Figura 5.2: Estrutura da base de datos



5.4.2 Plantilla

Debido a que utilizamos Django coma backend non é necesario o uso da plantilla para amosar os datos, esta responsabilidade recae no Frontend.

5.4.3 Vista

Na arquitectura MVT é a vista a encargada de executar a lóxica de negocio. Recibe as peticións dende o arquivo *urls.py* e executa as accións necesarias, no noso caso dividimos as peticións entre as necesarias para administrar os usuarios e as relacionadas coa música, dividindo esta última de novo entre as peticións a nosa API e as da API de Spotify.

User views:

debido a que utilizamos o modelo de usuario de Django modificado é este o encargado do manexo de case todas as peticións, nos tan só tivemos que engadir a obtención da sesión do usuario coma podemos ver en 5.4.3.

```
1 @api_view(['POST'])
2 def getInstance(request):
3     user =
    CustomUser.objects.filter(username=request.data['username']).values()
4     userInstance = user[0]['userInstance_id']
5     return Response(data=userInstance, status=200)
```

Api views:

as funcións explicadas a continuación son algunhas das mais destacables das solicitudes que se fan a xestión da nosa API, todas as peticións son de tipo POST debido a que sempre temos que saber cal é o usuario que fai a solicitude.

- **PopTop** é a función encargada de borrar a primeira canción da lista se é a que está a soar.

```
1  def popTop(wike):
2      current =
       SongPlayingNow.objects.filter(instance=wike).values('uri').first()
3      top =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
4      if current != None:
5      if current == top:
6          res = "same"
7
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').first().delete()
8      else:
9          res = "notSame"
10     else:
11         res = "Not playing"
12     return res
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify, comproba que falten menos de 30 segundos da canción que está a soar e engade a seguinte canción da lista local de cancións. Utiliza unha variable global xa que é necesario saber cal foi a última canción engadida para non engadila mais dunha vez.

```
lastInQueue = None # Global var for keep the last song added to the queue
def addToQueue(wike):
    totalTime = SongPlayingNow.objects.filter(instance=wike).values('duration').first() # Get the total duration of the song
    nowTime = spotify.currentProgress(wike) # Get the progress of the song
    if (totalTime != None) and (nowTime != None): # Check that there are a song playing
        totalTime = totalTime['duration']
        nowTime = timedelta(milliseconds=nowTime['progress_ms'])
        changeTime = nowTime + timedelta(seconds=30) # Establish the time to add the next song
        if changeTime > totalTime:
            adding = LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').values('uri').first()
            global lastInQueue
            if adding != lastInQueue: # Check if the last added song is the same as the one at the top of the LocalPlaylist
                spotify.addToQueue(wike)
                lastInQueue = LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').values('uri').first()
                res = "Change"
            else:
                res = "Already added"
        else:
            res = "No change"
    else:
        res = "No playing"

    return res
```

- **VoteP** é a función de votación, permite ao usuario engadir un voto novo a canción elixida.

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def voteP(request):
    N = LocalPlaylist.objects.values_list('votes', flat=True).get(instance=request.user.userInstance, uri=request.data['uri'])
    N = N+1
```

```
6
    LocalPlaylist.objects.filter(instance=request.user.userInstance).filter(uri=request.d
7   return Response(data="plus", status=200)
```

- **CheckerThread** para implementar a funcionalidade de que o administrador non teña que ter a páxina da aplicación aberta e as comprobacións para detectar se a canción vaise acabar ou se ten que borrar a seguinte, creamos unha función que executa ditas comprobacións cada 10 segundos. Esta función execútase nun fío paralelo, xa que Django ten que poder responder ao resto das peticións e desta forma evitamos o bloqueo.

```
1  def checkerThread():
2      while True:
3          wikes = instance.objects.all()
4          for wike in wikes:
5              if spotify.areToken(wike):
6                  if spotify.currentProgress(wike) != None and
       spotify.currentProgress(wike)['is_playing']:
7                      addToQueue(wike)
8                      popTop(wike)
9          time.sleep(10)
10
11 x = threading.Thread(target=checkerThread)
12 x.start()
```

Spotify views:

o conxunto seguinte de funcións son as relacionadas coas peticións a API de Spotify, algunhas son simplemente a petición directamente, por isto só amosamos as mais destacables.

- **Amanager** esta función é a encargada de xerar o xestor de autenticación de Spotify.

```
1  def amanager(userInstance):
2      instance = str(userInstance)
3      auth_manager = spotipy.oauth2.SpotifyOAuth(client_id=client_id,
       client_secret=client_secret, redirect_uri=redirect_uri,
       scope=scope, cache_path=".spotify_token_" + instance)
4      return auth_manager
```

- **ChooseDevice** permite ao administrador elixir en que dispositivo quere que se reproduza a música no caso de que haxa mais de un.

```
1  def chooseDevice(userInstance):
2      client = getClient(userInstance)
3      devices = client.devices()
```

```
    devices = devices['devices']
    devicesDict = {}
    # If no active devices send no active devices
    if len(devices) >= 1:
        for i in range(len(devices)):
            devicesDict[i] = {
                    'id': devices[i]['id'],
                    'name': devices[i]['name']
                    }
    else:
        devicesDict[0] = {
                'id': "No id",
                'name': "No active devices"
                }

    devicesJson = json.dumps(devicesDict)
    return devicesJson
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify.

```
def addToQueue(userInstance):
    client = getClient(userInstance)
    curi =
    LocalPlaylist.objects.filter(instance=userInstance).order_by('-added_mode',
    '-votes', 'created').values('uri').first() # Get the uri of the
    next song
    ccuri = json.dumps(curi['uri'])
    cccuri = ccuri[15:]
    rcccuri = cccuri.replace("\"", "")
    try:
        client.add_to_queue(rcccuri, None)
    except:
        pass
```

- **Search** fai a procura das cancións para o usuario.

```
    def search(search, userInstance):
        client = getClient(userInstance)
        searchResponse = client.search(search, limit=10) # Retuns a
    'dict' of songs
        cleanlist = searchResponse['tracks']['items'] # Gets the
    important part of the response
        # Creates a dictionary and populates it with the responses
        songList = {}
        for i in range(len(cleanlist)):
            songList[i] = {
```

```
                    'song': cleanlist[i]['name'],
                    'artists': cleanlist[i]['artists'][0]['name'],
                    'uri': cleanlist[i]['uri']
                    }
        songJson = json.dumps(songList) # Convert the dictionary to
    a json
        return songJson
```

- **SearchToLocalPlaylist** engade a canción seleccionada ao modelo de datos.

```
def searchToLocalPlaylist(songUri, userInstance):
    client = getClient(userInstance)
    playlist = LocalPlaylist()  # Assign the model
    searchedSong = client.track(songUri)
    playlist.pk = None  # The pk is assigned automatically, this is
    needed for that
    artists = searchedSong['artists']  # Loop in the artist if
    there are more than one
    string = ""
    for j in range(len(artists)):
        string += artists[j]['name']
        if j+1 != len(artists):
            string += ", "  # Separate the artist names
    playlist.artists = string
    playlist.title = searchedSong['name']
    playlist.duration = datetime.timedelta(milliseconds =
    searchedSong['duration_ms'])
    playlist.uri = searchedSong['uri']
    playlist.created = datetime.datetime.now()
    playlist.added_mode = "search"
    playlist.instance = userInstance  # The instance of the user
    comes as parameter
    playlist.save()
    return json.dumps(searchedSong)
```

Capítulo 6

Conclusións e traballo futuro

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras.

6.1 Conclusións

O sistema creado cumpre os requisitos desexados, creamos unha gramola virtual que permite aos usuarios o uso mais básico de estas, que é a reprodución das cancións preferidas nunha orde. Ademais de ser un sistema o mais sinxelo posible o que facilita o uso xa que con poucas iteracións do usuario pódese crear unha sesión e reproducir música en calquera dispositivo con Spotify de forma colaborativa.

6.2 Traballo futuro

Coma calquera proxecto software, sempre cabe a posibilidade da implementación de novas ideas que melloren o produto final, no noso caso a lista podería ser moi longa xa que existen moitos sistemas diferentes para a reprodución de música e a inclusión de algúns destes melloraría o valor da nosa aplicación. Nun principio a aplicación non busca o lucro, aínda que se poderían poñer en funcionamento algunhas funcionalidades para que así fose. A continuación expoñemos algunhas das ideas que se poderían desenvolver para a mellora da aplicación, estas son sempre opcións non exclusivas, polo que cun panel de axustes o administrador podería elixir a súa configuración desexada.

- **Modo DJ:** cun funcionamento similar, a aplicación non engadiría as cancións automaticamente a cola, mostraría unha lista a unha persoa que se dedique a pinchar segundo os votos dos usuarios e as súa propia elección.

- **Sistema de créditos:** limitar o número de cancións por usuario e ofrecer un código para conseguir aumentar o límite, por exemplo nun negocio de restauración para incitar o consumo dos clientes.
- **Usuarios globais:** poder ter o mesmo usuario para diferentes sesións en diferentes lugares. Deste xeito poderíase manter un avatar e unhas estadísticas de uso persoais.
- **Estadísticas:** amosar aos usuarios estadísticas da sesión, cancións mais reproducidas, usuarios que mais cancións engaden...
- **Gardar listas:** as cancións que soen nun período de tempo gardalas nunha lista de reprodución de Spotify, deste xeito poderíase gardar por exemplo a lista de reprodución dunha noite nun bar.

Apéndices

Bibliografía